

**Escuela Politécnica Nacional
Facultad De Ingeniería De Sistemas
Ingeniería De Software
Construcción Y Evolución De Software
(ISWD633)
GR2SW**



**Proyecto: Documento de Construcción y
Evolución de Software**

**DocuSigner Enterprise
Calvache Mateo**

31/01/2026

1. Introducción

Descripción del Problema

Actualmente, la empresa "Logística y Servicios del Ecuador" gestiona sus contratos de prestación de servicios y actas de entrega-recepción de forma manual. Este proceso implica la impresión física de documentos, la coordinación para la firma presencial de los involucrados y el posterior escaneo para su archivo digital. Este flujo genera un cuello de botella operativo, con un retraso promedio de 3 días laborables por trámite, riesgo elevado de pérdida de documentos físicos y un gasto recurrente e innecesario de papel e insumos de oficina.

Descripción del Proyecto (Solución)

El proyecto **DocuSigner Enterprise** es una plataforma web centralizada diseñada para la digitalización completa del ciclo de vida de los documentos corporativos. El sistema permite a los administradores cargar plantillas de contratos en formato PDF, asignar destinatarios específicos y facilitar la firma electrónica de los documentos. El software garantiza la integridad de los archivos, gestiona el almacenamiento seguro en la nube y notifica automáticamente a las partes interesadas, eliminando la necesidad de gestión física.

Objetivo del Documento

El objetivo de este documento es definir formalmente cómo se gestionará la **construcción y evolución del software**. Se detallan la arquitectura técnica, las estrategias de integración y despliegue continuo (CI/CD), el flujo de trabajo colaborativo en el repositorio y las herramientas de gestión que garantizarán un desarrollo ordenado, escalable y mantenible.

2. Arquitectura del Proyecto

Descripción de Componentes

El sistema sigue una arquitectura web moderna basada en capas para desacoplar la lógica de negocio de la interfaz de usuario.

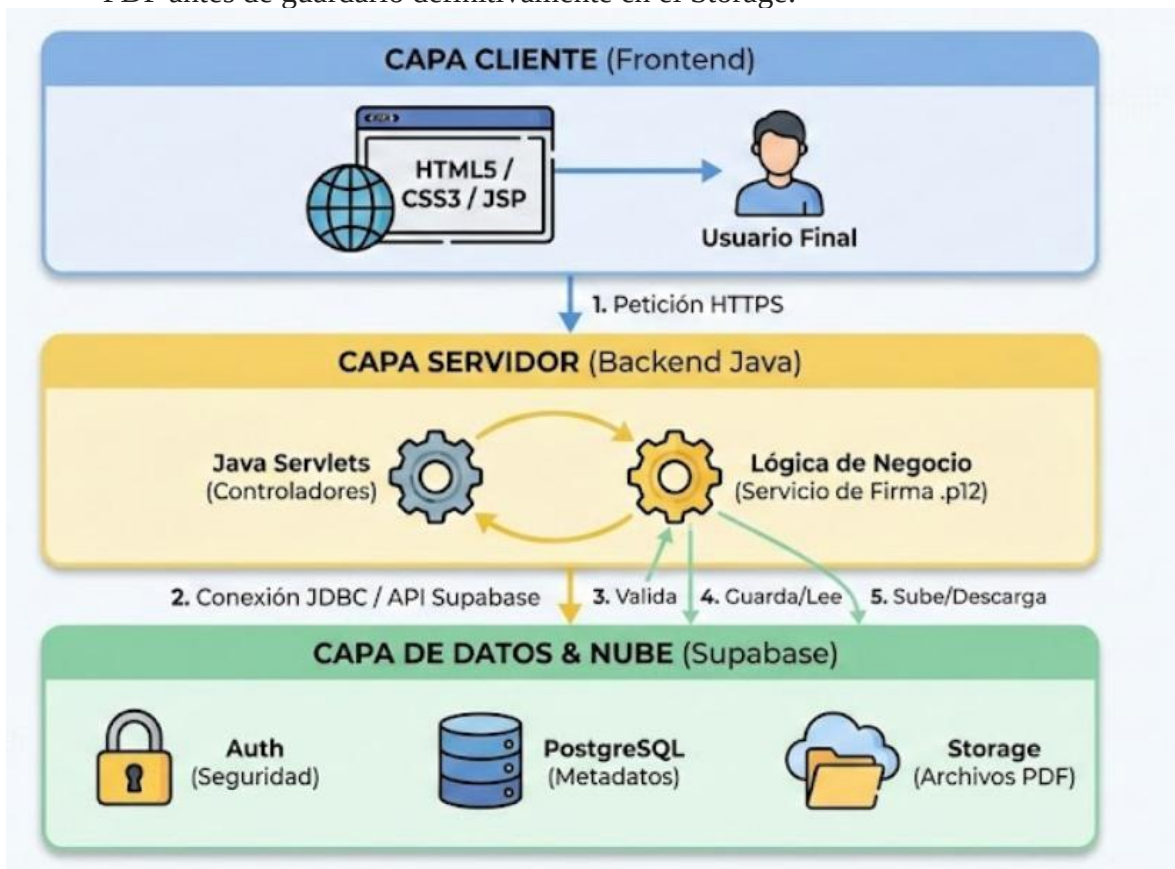
1. **Frontend (Interfaz de Usuario):** Desarrollado con tecnologías web (HTML5, CSS3, JavaScript/JSP) que proveen una interfaz responsiva para la carga y visualización de documentos.
2. **Backend (Lógica de Negocio):** Implementado en Java (Servlets/MVC). Se encarga de la gestión de usuarios, la validación de permisos y el procesamiento de las firmas digitales.
3. **Base de Datos y Almacenamiento:** Se utiliza **Supabase (PostgreSQL)**.
 - **Database:** Para almacenar metadatos de los documentos (IDs, estados, fechas, firmantes).

- **Storage:** Para el almacenamiento seguro de los archivos PDF físicos (versiones originales y firmadas).

Estrategia de Integración

La comunicación entre módulos es síncrona mediante protocolo HTTP/HTTPS:

- El **Frontend** envía peticiones al Backend mediante formularios y llamadas AJAX.
- El **Backend** procesa la solicitud y se conecta con Supabase mediante su librería cliente (o JDBC) para persistir la información.
- El sistema de **Firma** se integra como un servicio dentro del backend que manipula el PDF antes de guardarlo definitivamente en el Storage.



3. Estrategia de Pipelines (CI/CD)

Se implementará una estrategia de automatización utilizando **GitHub Actions** para asegurar que el código nuevo no rompa la funcionalidad existente.

Pipeline de Integración Continua (CI)

Este pipeline se ejecuta automáticamente ante cada `push` o `pull request` hacia las ramas de desarrollo.

1. **Linting:** Verificación de estilo de código Java para asegurar legibilidad.
2. **Compilación:** El sistema intenta compilar el proyecto (Build) para detectar errores de sintaxis.
3. **Pruebas Unitarias:** Ejecución de tests automatizados (JUnit) para validar la lógica crítica (ej. verificar que un usuario sin permisos no pueda firmar).

Pipeline de Entrega Continua (CD)

Este pipeline se activa únicamente cuando se fusionan cambios aprobados a la rama `main` (Producción).

1. **Empaquetado:** Generación del archivo `.war` o contenedor final.
2. **Despliegue (Deploy):** Publicación automática en el entorno de pruebas/producción para que los cambios estén disponibles inmediatamente para los usuarios.

4. Estrategia de Flujos de Desarrollo

Se utilizará el modelo de ramificación **Gitflow** para organizar el trabajo del equipo y separar el código estable del código en desarrollo.

Modelo de Ramas

- **main:** Contiene únicamente la versión estable y lista para producción del software.
- **develop:** Rama principal de integración donde se unen las nuevas funcionalidades.
- **feature/*:** Ramas temporales creadas para desarrollar una Historia de Usuario específica (ej. `feature/carga-documentos`).
- **hotfix/*:** Ramas exclusivas para corregir errores críticos detectados en producción.

Descripción del Flujo

Cada nueva funcionalidad comienza creando una rama `feature/` desde `develop`. Al finalizar la tarea, el desarrollador abre un **Pull Request** hacia `develop`. Una vez aprobado y fusionado, y cuando se tiene un conjunto de funcionalidades listas, se prepara el paso de `develop` a `main` para una nueva versión (release).

5. Gestión de Historias de Usuario

Las funcionalidades del sistema se definen y priorizan utilizando el formato estándar de Historias de Usuario.

Formato

"Como [rol], quiero [funcionalidad], para [beneficio]."

Ejemplos del Proyecto

1. Carga de Documentos:

- Como Asistente Administrativo, *quiero* subir un archivo PDF y asignar un correo electrónico de destinatario, *para* iniciar el proceso de firma sin necesidad de imprimir el documento.

2. Firma Electrónica:

- Como Gerente, *quiero* visualizar los documentos pendientes y aprobarlos con un clic, *para* agilizar los trámites desde cualquier ubicación.

3. Historial de Cambios:

- Como Auditor, *quiero* ver un registro de fechas y horas de cada firma, *para* garantizar la trazabilidad del proceso.

Gestión en Tablero (Jira/Trello)

Cada historia se convierte en un ticket que pasa por los estados: *Por hacer (Backlog)* -> *En Progreso* -> *En Revisión (QA)* -> *Finalizado*.

6. Estrategia de Revisiones y Aprobaciones

La calidad del código se asegura mediante revisiones obligatorias antes de cualquier integración.

Pull Requests (PRs)

- Ningún código entra a la rama `develop` o `main` directamente. Todo debe pasar por un Pull Request.
- El PR debe incluir una descripción clara de qué problema resuelve.
- Se requiere la aprobación de **al menos un miembro del equipo** (diferente al autor) para poder fusionar (merge).

Checklist de Revisión

El revisor debe validar:

- ☐ El código cumple con los estándares definidos.
- ☐ No existen credenciales o claves de API expuestas en el código.
- ☐ La funcionalidad corresponde a lo solicitado en la Historia de Usuario.
- ☐ El código es legible y cuenta con comentarios donde la lógica es compleja.

7. Herramientas y Conexiones

El ecosistema de desarrollo integra herramientas de gestión, versionamiento y comunicación:

- **Gestión de Tareas: Trello / Jira.** Se utiliza para visualizar el avance del Sprint.
- **Repositorio de Código: GitHub.** Aloja el código fuente y ejecuta los pipelines de CI/CD.
- **Comunicación: Slack / Teams.** Canales dedicados para notificaciones automáticas.

Ejemplo de Integración

Cuando un desarrollador hace un commit en GitHub, incluye el código del ticket (ej. #DOC-101). Esto permite que Jira/Trello detecte el cambio y enlace el código con la tarea. Al crearse un Pull Request, se envía una notificación automática al canal de Slack del equipo avisando que se requiere una revisión.

8. Conclusiones

La implementación de estas estrategias de construcción y evolución proporciona una base sólida para el desarrollo de **DocuSigner Enterprise**. La arquitectura desacoplada facilita el mantenimiento y la escalabilidad futura (como añadir nuevos métodos de firma). El uso estricto de Gitflow y revisiones de código minimiza la deuda técnica y los errores en producción. Finalmente, la automatización mediante CI/CD permite al equipo enfocarse en desarrollar valor para el usuario, reduciendo el tiempo dedicado a tareas manuales de despliegue.